Product Requirements Document

Couple Finance Tracker Web Application

1. Project Overview

The goal of this project is to transition from a static spreadsheet to a dynamic web application for managing shared household finances. The application will allow two users to log in, input variable monthly incomes and expenses, and automatically calculate financial distributions based on a specific custom logic.

2. Recommended Tech Stack

To ensure a robust, modern, and easily deployable application, the following stack is recommended for development:

- **Frontend Framework:** Next.js 15 (App Router) with TypeScript.
- **Styling:** Tailwind CSS & Framer Motion for a clean, responsive UI.
- **Backend & Database:** Supabase (PostgreSQL).
- **Authentication:** Supabase Auth using Google OAuth.

3. Core Features

3.1 Authentication & User Management

- Google OAuth integration for quick and secure login.
- Role/Access management restricted to the two primary users (Jairo and Naroa).

3.2 Data Input (Monthly Scope)

Each month, users will input or confirm the following data points:

- **Incomes:** Variable monthly salaries for both users.
- **Fixed Expenses:** Pre-filled from user settings but editable (e.g., Joint Account fixed at €1480, Groceries at €600, personal fixed costs).
- **Variable/Extra Expenses:** One-off expenses assigned to specific months.
- **Credit Card:** A specific input for the monthly credit card bill.

3.3 Calculation Engine Logic

The application must execute the following financial logic automatically upon data entry:

> **Step 1: Calculate Total Liability**
>
> `Total Expenses = Joint Fixed + Joint Groceries + User1_Fixed + User2_Fixed + Extra + CreditCard`

> **Step 2: Calculate User 1 (Primary Payer) Position**
>
> `User1_Balance = User1_Income - Total Expenses`
>
> *Note: User 1 pays all bills from their income first.*

Step 3: Calculate Shortfall & User 2 Distribution

```
If User1_Balance < 0: (There is a shortfall)
  Shortfall = Absolute(User1_Balance)
  User2_Remaining = User2_Income - Shortfall

If User1_Balance >= 0: (No shortfall)
  User2_Remaining = User2_Income

Distribution of User2_Remaining:
  Savings = User2_Remaining * 0.50
  User1_Payout = User2_Remaining * 0.25
  User2_Payout = User2_Remaining * 0.25
```

3.4 Historical Data & Dashboard

- A dashboard displaying the current month's breakdown, who owes what to whom, and how much goes to savings.
- Historical view to look back at previous months' incomes, expenses, and savings rates stored in the database.

4. Database Schema Proposal

Table Name	Description
`users`	Stores authenticated users (Google Auth integration).
`monthly_records`	ID, month, year, user1_income, user2_income, credit_card_bill, locked_status.
`expenses`	Linked to monthly_records. Fields: type (fixed, extra, joint), amount, description, assignee.